

Mantenimiento de Software

CRUD pets-crud

Aplicación de mantenimiento correctivo, adaptativo, perfectivo y preventivo sobre un CRUD en Node.js + Express + SQLite.

Node.js

Express

SQLite

API-Key

Logs

Estado actual del CRUD pets-crud

El sistema ya tiene una base funcional, pero necesita mejoras.



Lo que ya existe

- Express + SQLite.
- CRUD de mascotas.
- Frontend web.
- Validaciones básicas.
- Pruebas iniciales.



Problemas detectados

- Logs no estructurados.
- Sin API-Key.
- Sin .env formal.
- Faltan try-catch.
- Riesgo XSS en frontend.



Deuda técnica

- Pruebas limitadas.
- API sin documentación formal.

• punto de partida

```
const PORT = process.env.PORT || 4002;
```

```
app.get('/pets', async (req, res) => {  
  const pets = findAllPets();  
  return res.json(pets);  
});
```



Conclusión del diagnóstico

El CRUD sirve como base, pero se debe fortalecer la seguridad, trazabilidad, manejo de errores, validación de datos, pruebas y documentación.

Ruta técnica de trabajo

Orden recomendado para aplicar los cambios sin romper el proyecto.



Correctivo: trazabilidad y corrección documental

Se corrigen debilidades que afectan el diagnóstico y seguimiento del sistema.



Problema

Los mensajes actuales usan
console.log/console.error.

No hay niveles formales INFO, WARN, ERROR.



Cambio aplicado

utils/logger.js

logs/app.log

Formato JSON con timestamp ISO.

ejemplo de log

```
{
  "timestamp": "2026-05-
29T12:20:00.000Z",
  "level": "INFO",
  "message": "PET_CREATED",
  "id": "PET-0001"
}
```

```

YO-SOY-CRUD
├── YO-SOY-CRUD
├── .github
├── audiobook-crud
├── epr-event-manager
├── esports-crud
├── missions-crud
├── pets-crud
├── db
├── logs
├── app.log
├── node_modules
├── public
├── test
├── pet-validation.test.js
├── pets-api.test.js
├── utils
├── .env
├── .env.example
├── .gitignore
├── API_DOCS.md
├── eslint.config.mjs
├── package-lock.json
├── package.json
├── README.md
├── server.js
├── .gitignore
├── AGENTS.md
├── README.md
├── .gitignore

```

```

1 {"timestamp": "2026-05-29T07:08:01.552Z", "level": "WARN", "message": "API_KEY_INVALID_OR_MISSING", "method": "GET", "path": "/pets", "ip": "::ffff:127.0.0.1"}
2 {"timestamp": "2026-05-29T07:08:01.688Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "QUERY", "petId": "ALL"}
3 {"timestamp": "2026-05-29T07:08:01.688Z", "level": "INFO", "message": "PETS_LISTED", "total": 0}
4 {"timestamp": "2026-05-29T07:08:01.645Z", "level": "WARN", "message": "PET_VALIDATION_ERROR", "operation": "CREATE", "errors": [{"name es obligatorio", "species es obligatorio", "species inv"}
5 {"timestamp": "2026-05-29T07:08:01.656Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "CREATE", "petId": "PET-0001"}
6 {"timestamp": "2026-05-29T07:08:01.656Z", "level": "INFO", "message": "PET_CREATED", "id": "PET-0001", "name": "Firula's Test", "species": "perro"}
7 {"timestamp": "2026-05-29T07:08:01.664Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "DELETE", "petId": "PET-0001"}
8 {"timestamp": "2026-05-29T07:08:01.664Z", "level": "INFO", "message": "PET_DELETED", "id": "PET-0001", "name": "Firula's Test"}
9 {"timestamp": "2026-05-29T07:09:01.849Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "QUERY", "petId": "ALL"}
10 {"timestamp": "2026-05-29T07:09:01.851Z", "level": "INFO", "message": "PETS_LISTED", "total": 0}
11 {"timestamp": "2026-05-29T07:10:12.178Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "CREATE", "petId": "PET-0001"}
12 {"timestamp": "2026-05-29T07:10:12.180Z", "level": "INFO", "message": "PET_CREATED", "id": "PET-0001", "name": "Duffy", "species": "perro"}
13 {"timestamp": "2026-05-29T07:10:12.208Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "QUERY", "petId": "ALL"}
14 {"timestamp": "2026-05-29T07:10:12.210Z", "level": "INFO", "message": "PETS_LISTED", "total": 1}
15

```

```

PS C:\Users\User\Documents\Construccion\ExamenCRUD\YO-SOY-CRUD\YO-SOY-CRUD\pets-crud> npm test
PS C:\Users\User\Documents\Construccion\ExamenCRUD\YO-SOY-CRUD\YO-SOY-CRUD\pets-crud> npm start
> pets-crud@1.0.0 start
> node server.js

◊ Injected env (S) from .env // tip: @secrets for agents [see.domeva.com]
◊ Sistema de Gestión de Mascotas corriendo en http://localhost:3000
Base de datos: ./db/pets.sqlite
# Enviando eventos al Event Manager en http://localhost:3000
{"timestamp": "2026-05-29T07:09:01.849Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "QUERY", "petId": "ALL"}
{"timestamp": "2026-05-29T07:09:01.851Z", "level": "INFO", "message": "PETS_LISTED", "total": 0}
{"timestamp": "2026-05-29T07:10:12.178Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "CREATE", "petId": "PET-0001"}
{"timestamp": "2026-05-29T07:10:12.180Z", "level": "INFO", "message": "PET_CREATED", "id": "PET-0001", "name": "Duffy", "species": "perro"}
{"timestamp": "2026-05-29T07:10:12.208Z", "level": "INFO", "message": "EVENT_MANAGER_SENT", "action": "QUERY", "petId": "ALL"}
{"timestamp": "2026-05-29T07:10:12.210Z", "level": "INFO", "message": "PETS_LISTED", "total": 1}

```

Adaptativo: seguridad y configuración externa

El CRUD se adapta a una regla institucional: consumir la API solo con una clave autorizada.



API-Key

Header obligatorio:

X-FIS-EPN-KEY

Sin clave → 401 Unauthorized.



.env

- PORT
- PETS_DB_PATH
- EVENT_MANAGER_URL
- API_KEY



Frontend

fetch envía header.

Se mantiene funcional.

Health queda libre.

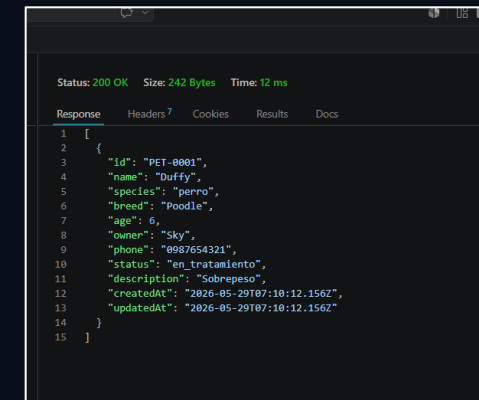
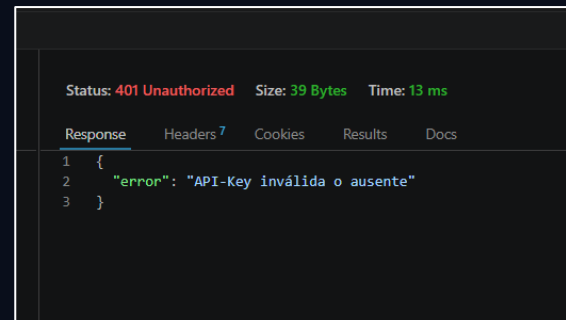


middleware de seguridad

```
function requireApiKey(req, res, next) {
  const receivedKey = req.header('X-FIS-EPN-KEY');

  if (!receivedKey || receivedKey !== API_KEY) {
    return res.status(401).json({
      error: 'API-Key inválida o ausente'
    });
  }

  return next();
}
```



Perfectivo: pruebas y documentación de API

Se mejora la mantenibilidad del sistema y se facilita su revisión técnica.



Pruebas existentes

validatePet()
normalizeStatus()
normalizeSpecies()



Pruebas nuevas

GET /pets sin API-Key → 401
GET /pets con API-Key → 200
POST inválido → 400
Crear y eliminar mascota válida.



Documentación

API_DOCS.md
Headers requeridos.
Ejemplos request/response.
Códigos de error.



comandos de validación

npm run check
npm run lint
npm test

```
PS C:\Users\User\Documents\Construccion\ExamenCRUD\VO-SOY-CRUD\VO-SOY-CRUD\pets-crud> npm run check
PS C:\Users\User\Documents\Construccion\ExamenCRUD\VO-SOY-CRUD\VO-SOY-CRUD\pets-crud> npm run lint

> pets-crud@1.0.0 lint
> eslint .

PS C:\Users\User\Documents\Construccion\ExamenCRUD\VO-SOY-CRUD\VO-SOY-CRUD\pets-crud> npm test

> pets-crud@1.0.0 test
> node --test test/*.test.js

◇ injected env (5) from .env // tip: secrets for agents [www.dotenvx.com]
✓ validación: rechaza mascota sin nombre, especie y dueño (0.6116ms)
✓ validación: acepta una mascota correcta (0.4828ms)
✓ estado: normaliza estados desconocidos a activo (0.1997ms)
✓ especie: normaliza especies desconocidas a otro (0.1096ms)
◇ injected env (4) from .env // tip: suppress logs { quiet: true }
{"timestamp":"2026-05-29T07:08:01.552Z","level":"WARN","message":"API_KEY_INVALID_OR_MISSING","method":"GET","path":"/pets","ip":"::ffff:127.0.0.1"}
✓ seguridad: rechaza GET /pets sin API-Key (20.6158ms)
{"timestamp":"2026-05-29T07:08:01.608Z","level":"INFO","message":"EVENT_MANAGER_SENT","action":"QUERY","petId":"ALL"}
{"timestamp":"2026-05-29T07:08:01.608Z","level":"INFO","message":"PETS_LISTED","total":0}
✓ seguridad: permite GET /pets con API-Key correcta (48.6884ms)
{"timestamp":"2026-05-29T07:08:01.645Z","level":"WARN","message":"PET_VALIDATION_ERROR","operation":"CREATE","errors":[{"name es obligatorio","species es obligatorio","species inválida","owner es obligatorio"}]}
✓ validación API: rechaza crear mascota incompleta (37.017ms)
{"timestamp":"2026-05-29T07:08:01.656Z","level":"INFO","message":"EVENT_MANAGER_SENT","action":"CREATE","petId":"PET-0001"}
{"timestamp":"2026-05-29T07:08:01.656Z","level":"INFO","message":"PET_CREATED","id":"PET-0001","name":"Firula's Test","species":"perro"}
{"timestamp":"2026-05-29T07:08:01.664Z","level":"INFO","message":"EVENT_MANAGER_SENT","action":"DELETE","petId":"PET-0001"}
{"timestamp":"2026-05-29T07:08:01.664Z","level":"INFO","message":"PET_DELETED","id":"PET-0001","name":"Firula's Test"}
✓ CRUD API: crea y elimina una mascota válida (18.552ms)

i tests 8
i suites 0
i pass 8
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 357.851

PS C:\Users\User\Documents\Construccion\ExamenCRUD\VO-SOY-CRUD\VO-SOY-CRUD\pets-crud>
```

Preventivo: evitar fallos futuros

Se refuerza el sistema antes de que los errores lleguen al usuario final.



Validación reforzada

- Edad entre 0 y 50.
- Teléfono con formato válido.
- Campos obligatorios.
- Longitudes máximas.



Manejo de errores

- try-catch en endpoints.
- Respuestas 400/401/404/500.
- Servidor no se cae ante errores.

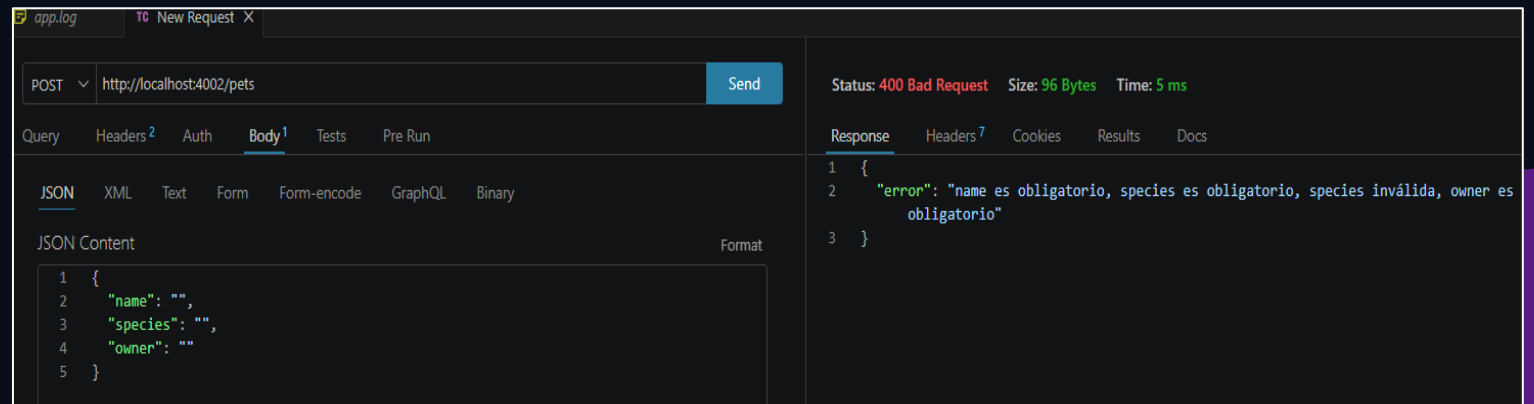


Protección XSS

- escapeHtml() en frontend.
- No ejecutar HTML ingresado por usuario.
- Renderizado más seguro.

sanitización en frontend

```
function escapeHtml(value) {  
  return String(value ?? '')  
    .replaceAll('&', '&amp;')  
    .replaceAll('<', '&lt;')  
    .replaceAll('>', '&gt;')  
    .replaceAll('"', '&quot;')  
    .replaceAll("'", '&#039;');  
}
```



Conclusión

El CRUD queda más seguro, mantenible, probado y preparado para evitar fallos futuros.



Seguro

API-Key obligatoria para endpoints /pets.



Trazable

Logs estructurados para operaciones y errores.



Probado

Pruebas automáticas para seguridad, validación y CRUD.



Preventivo

Validaciones, try-catch y protección frente a XSS.

“El mantenimiento no solo corrige errores: también adapta, mejora y previene.”